

Mandarin Journey

汉语之旅

Build Documentation — architecture, structure & decisions

Version 0.2 · HSK 1 + 2 · 30 units / 180 lessons
github.com/obedkm/chinese101
July 4, 2026 · built with Claude Code

1. What this app is

Mandarin Journey is a beginner Chinese course (HSK 1 and HSK 2) packaged as a small daily habit. The course is a linear journey of 30 themed units; every unit has the same six-lesson rhythm — New Words, Hanzi, Grammar, Listening, Speaking, Checkpoint — and each lesson takes three to five minutes. A spaced-repetition system tracks every word, a streak and a daily goal reward consistency, and a progress screen answers the two questions that matter: how many words do I know, and how many lessons did I finish each day?

Everything runs offline on the user's device. There is no account, no server, and no analytics; all progress lives in the browser's localStorage (inside the Android app, the WebView's storage).

Key product decisions

Decision	Choice	Why
Scope	HSK 1+2: 285 words, 120 hanzi, 30 grammar points	A complete, recognized beginner curriculum with a realistic 5–6 month journey.
Habit design	Daily goal + streak + one 'Continue' button	Zero decisions at open time; the app always knows the next 4-minute step.
UI language	English	Teach Chinese from English; interface stays minimal.
Platform	Static web app wrapped in an Android APK	The build machine has no Node or Android tooling, so the app avoids all local build steps and CI does the packaging.
AI	None in v1 (DeepSeek slot reserved)	Core lessons must work offline and never depend on an API.

2. Architecture

One codebase, three run targets. The web app is plain HTML/CSS/JS with no framework and no bundler — every file is loaded directly by script tags, so a static file server is a full dev environment. The same files are copied verbatim into the Android wrapper's assets at CI time.

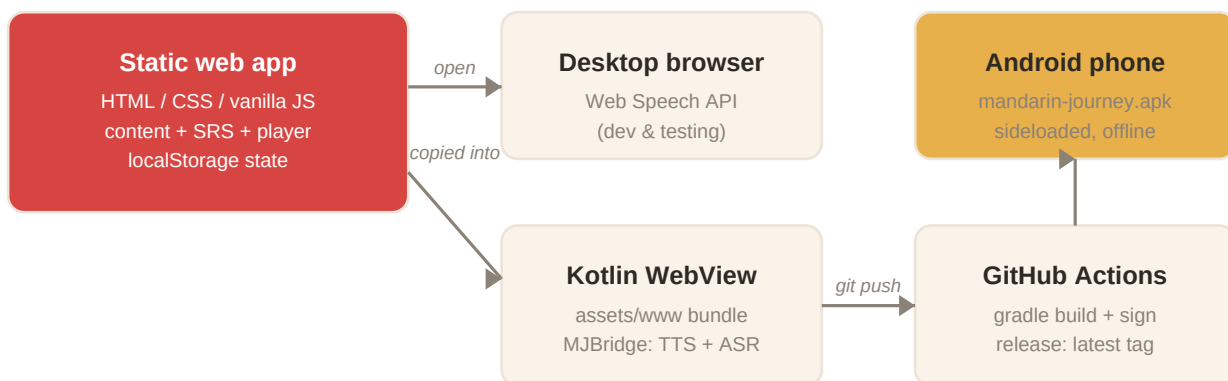


Figure 1 — the web app is developed and tested in a desktop browser; CI copies it into a thin Kotlin WebView shell and publishes a signed APK.

The only platform-specific code is speech. `js/speech.js` feature-detects its backend: in a browser it uses the Web Speech API (`speechSynthesis` + `webkitSpeechRecognition`); inside the APK it finds `window.MJBridge` — a JavascriptInterface exposed by the Kotlin wrapper — and routes text-to-speech and speech-recognition through Android's native engines instead. The rest of the app is identical on every target.

3. Repository structure

chinese101/	
index.html	app shell: 4 screens + lesson player overlay
css/style.css	single stylesheet, CSS variables, mobile-first (max 520px)
js/content-hsk1.js	WORDS + UNITS u1-u15 (HSK 1)
js/content-hsk2.js	extends WORDS, pushes u16-u30 (HSK 2)
js/srs.js	localStorage store + spaced-repetition + streak math
js/speech.js	TTS/ASR dual backend + pinyin normalizer
js/app.js	screens, lesson player, 9 exercise engines
js/hanzi-data.js	embedded stroke data for all 120 characters
vendor/hanzi-writer.min.js	HanziWriter 3.7 (stroke animation & tracing quiz)
data/<char>.json	cached per-character stroke JSONs (build source)
android/	Kotlin WebView wrapper (Gradle project)
keystore.p12	committed signing key -> stable APK signature
app/src/main/java/.../MainActivity.kt	
.github/workflows/build-apk.yml	CI: build, sign, publish release
docs/BUILD.pdf	this document

Script load order matters and replaces a module system: hanzi-writer, hanzi-data, content-hsk1, content-hsk2, speech, srs, app. Each file publishes one global (WORDS, UNITS, HANZI_DATA, Speech, Pinyin, Store, SRS); app.js is an IIFE that reads them all.

4. The web app, layer by layer

4.1 Content model

All course data is declarative. A word is one entry in WORDS; a unit bundles word ids, four characters, one grammar point (with fill-blanks and tile-ordering exercises), and four practice sentences used by listening and speaking lessons:

```
nihao: { cn: '你好', py: 'nǐ hǎo', pyPlain: 'nihao', en: 'hello', emoji: '..' }

{ id: 'u7', title: 'Food & Drink', hsk: 1,
  words: ['chi', 'he', 'mifan', 'cai', 'cha', 'shui', ...],
  hanzi: ['吃', '喝', '水', '果'],
  grammar: { name: '想 & 要 - wanting things', intro: '...',
    examples: [...], fillBlanks: [...], tiles: [...] },
  sentences: [{ cn: '我想喝茶。', py: 'Wǒ xiǎng hē chá.', en: '...' }, ...] }
```

pyPlain is the toneless, ü → v form used to check typed answers. Word ids are unique across the course; homophones get tone-number suffixes (shi 是 / shi2 十, mai 买 / mai4 卖). Content invariants are validated in the browser: every fill-blank answer appears in its choices, every unit has 4 sentences with distinct translations, at least 4 distinct meanings for quiz distractors, no word id reused across units, no character taught twice, and stroke data exists for all 120 characters.

4.2 Lesson generation

Lessons are not authored one by one — buildLesson(unit, lessonIndex) composes them from the unit data at start time, so authoring one unit yields six lessons automatically. Nine exercise engines cover the five skills:

Engine	Skill	What the learner does
Intro card	vocab	Sees hanzi + pinyin + meaning + emoji, hears the audio.
Multiple choice	vocab	Picks the meaning of a hanzi word among 4 unit-mates.
Reverse choice	vocab	Picks the hanzi for an English meaning.
Pinyin typing	vocab	Types toneless pinyin; compared via the normalizer.
Stroke learn	hanzi	Watches animated stroke order (HanziWriter).
Stroke quiz	hanzi	Traces the character stroke by stroke; ≤5 mistakes passes.
Fill the blank	grammar	Completes a sentence from 3 choices.
Tile ordering	grammar	Arranges shuffled word tiles into a sentence.
Audio choice / Speak	listening / speaking	Hears a sentence and picks the meaning; or reads one aloud and the recognizer transcript is scored (≥60% character match passes).

The Checkpoint lesson samples all of the above and requires 70% accuracy; passing it unlocks the next unit. Lessons unlock sequentially inside a unit as well, which keeps the journey linear and decision-free.

4.3 Spaced repetition

Every graded answer about a word — in any lesson type — feeds a simplified Leitner system (SRS.record). A word sits in one of six boxes; correct answers promote, wrong answers demote, and the box sets the next review date:

Box	0	1	2	3	4	5
Next review	today	+1 day	+3 days	+7 days	+14 days	+30 days
Stage	new	learning	learning	known	known	mastered

The home screen offers a daily Review session built from words whose due date has arrived (up to 12, random mix of the four vocab engines). The headline metric 'words I know' counts stages known + mastered — a word must survive several spaced recalls before it counts, which keeps the number honest.

4.4 Habit loop & progress

State is one JSON blob under localStorage key `mj_state_v1`: settings (daily goal), completed lessons, a per-day session counter, per-word SRS state, learned characters, and per-skill lesson counts. The streak is computed, not stored: consecutive days with at least one session, ending today or yesterday. The progress screen renders words-known with a stage breakdown, a 30-day sessions bar chart with the goal line, streak / best streak, characters learned, units finished, and a skill-balance chart — all derived from that one blob.

4.5 Speech and pinyin

```
Speech.speak(text, rate)    -> MJBridge.speak(...)          (APK)
                             -> speechSynthesis zh-CN voice (browser)
Speech.listen(cb, onEnd)    -> MJBridge.startListening('zh-CN'), result arrives
                             via window._mjOnSpeech(text)      (APK)
                             -> webkitSpeechRecognition zh-CN  (browser)
```

Typed pinyin is normalized before comparison: lowercase, diacritics folded (`n ĭ h ǎ o` → `nihao`, `ǔ` → `u`), everything non-alphabetic dropped — so `'ni hao'`, `'nihao'` and `'ni3hao3'` all match. Speaking answers are scored by Chinese-character overlap between target and transcript, after mapping Arabic digits back to hanzi (recognizers return `'3'` for 三).

4.6 Hanzi engine and the stroke-data pipeline

Stroke animation and tracing use the open-source HanziWriter library, vendored as one file. Its per-character stroke definitions normally load from a CDN; to stay offline, a small Python step scans the content files for every taught character, downloads each missing JSON from `hanzi-writer-data` (cached under `data/`), and rebuilds `js/hanzi-data.js` — a single embedded object covering all 120 characters (~250 KB). The app's `charDataLoader` then never touches the network.

5. The course: 30 units

285 words across 30 units — 146 in HSK 1, 139 in HSK 2. Four characters per unit (120 total). Grammar follows a classic progression; sentences reuse previously taught vocabulary almost exclusively, so nothing in a lesson feels unearned.

Unit	Theme	HSK	Words	Hanzi	Grammar point
u1	Greetings	1	8	你好我是	是 sentences & 吗 questions
u2	Numbers 1–10	1	12	一 二 三 十	Counting things: 个 and 几
u3	Politeness	1	10	谢 再 见 请	Polite magic words
u4	Names & Nations	1	12	叫 名 中 国	什么 questions & 的 (my / your)
u5	Family	1	10	有 家 爱 岁	有 — to have (and 没有 — to not have)
u6	This & That	1	11	这 那 书 猫	这 this & 那 that
u7	Food & Drink	1	10	吃 喝 水 果	想 & 要 — wanting things
u8	Dates	1	10	天 今 年 月	Dates: 月, 号, 星期
u9	Clock & Routine	1	9	点 分 午 晚	Telling time with 点
u10	Places	1	10	在 哪 里 学	在 — being somewhere
u11	Doing Verbs	1	9	去 来 回 写	Verb + place, verb + object
u12	Weather & Feelings	1	9	不 也 热 冷	很 + adjective, 不 + anything
u13	Shopping	1	10	买 钱 大 小	多少钱? & 太…了!
u14	Getting Around	1	7	车 坐 开 飞	坐 & 开 — how you travel
u15	Can Do It!	1	9	会 能 电 话	会 & 能 — I can!
u16	Feelings	2	10	忙 累 休 息	因为…所以… (because… so…)
u17	Done!	2	8	已 到 完 起	了 — it happened!
u18	Directions	2	10	远 近 门 边	A 离 B 很远 — how far apart
u19	Travel	2	10	走 路 票 玩	怎么走? — asking the way
u20	Money Talk	2	10	贵 卖 两 百	Measure words & 两
u21	At the Restaurant	2	10	鱼 肉 蛋 奶	一点儿 — just a little
u22	Big Family	2	11	哥 姐 弟 妹	比 — comparing things
u23	Work & Study	2	10	问 课 考 懂	可以 — may I?
u24	Time Flies	2	9	时 间 每 等	每 — every single one
u25	Say It	2	9	给 别 知 事	正在 — happening right now
u26	Sports & Fun	2	8	球 踢 唱 跳	一起 — doing it together
u27	Colors & Looks	2	10	红 白 黑 高	Describing with 的
u28	How & Why	2	9	您 它 快 慢	吧 — come on, let's!
u29	Around the House	2	8	房 床 手 表	送给 — giving things to people
u30	Big Finale	2	7	希 望 意 思	等一下! & 最 — wrapping up

6. The Android wrapper

android/ is the smallest possible Gradle project: one Activity, no dependencies, no AndroidX. MainActivity creates a WebView (JavaScript and DOM storage enabled — DOM storage is where all progress lives), loads file:///android_asset/www/index.html, and exposes a two-method bridge:

Bridge call (from JS)	Native implementation
MJBridge.speak(text, rate)	android.speech.tts.TextToSpeech, locale zh-CN, QUEUE_FLUSH — reuses the device's installed Chinese voice.
MJBridge.startListening('zh-CN')	android.speech.SpeechRecognizer with a free-form zh-CN intent; first use triggers the RECORD_AUDIO runtime permission. The best transcript (or empty string on error/denial) is delivered back with evaluateJavascript calling window._mjOnSpeech(text).

Manifest notes: RECORD_AUDIO and INTERNET permissions, plus a <queries> entry for android.speech.RecognitionService (required on Android 11+ to discover the recognizer). minSdk 24 (Android 7), targetSdk 34, portrait-locked. The launcher icon is a hand-written vector drawable — no mipmap assets needed.

Signing

android/keystore.p12 is committed to the repo on purpose. It was generated locally with openssl (the Mac has no Java for keytool) and holds a 30-year self-signed RSA key. For a sideloaded personal app the key secures nothing — its only job is that every CI build carries the same signature, so a new APK installs over the old one without uninstalling (which would wipe localStorage and the learner's streak). The password is plain 'mandarin-journey' and is documented in the README. LibreSSL writes legacy PKCS12 ciphers, so the workflow has a normalize step that re-wraps the same key material with modern ciphers if Java refuses to read it.

7. CI/CD: push → APK

The build machine constraint (no Node, no Android SDK, no Java) moved all packaging to GitHub Actions. .github/workflows/build-apk.yml runs on every push to main:

#	Step	Detail
1	Checkout + Temurin JDK 17	Standard setup actions.
2	Normalize keystore	keytool probe; converts legacy PKCS12 via openssl -legacy only if needed (same key, same signature).
3	Copy web app into assets	index.html, css/, js/, vendor/ → android/app/src/main/assets/www (gitignored; exists only at build time).
4	Gradle 8.7 assembleRelease	AGP 8.4.2, Kotlin 1.9.24; release build signed by the committed keystore.
5	Publish	APK uploaded as a build artifact and attached to the rolling 'latest' release — a stable public download URL.

The result: editing a lesson is 'change a JS file, git push, wait three minutes, re-download mandarin-journey.apk from github.com/obedkm/chinese101/releases/latest'. Updates install over the previous version with progress intact.

8. Development workflow & verification

- Local dev: any static server over the repo root. On the build Mac specifically, the sandboxed preview server cannot read ~/Documents, so the project is rsynced to a scratchpad directory and served from there with a

tiny Python HTTP server.

- Automated checks, run in the real browser: a validator walks all 30 units enforcing the content invariants of §4.1 (zero errors at v0.2), and a driver plays lessons end-to-end — answering multiple-choice, typing pinyin, building tile sentences — to verify the player, feedback, SRS writes, and the completion/streak flow.
- Manual checks on device: hanzi tracing (needs a finger), speaking (needs a microphone), and TTS voice availability (Google TTS with the Chinese voice installed).

9. Design decisions worth remembering

- No framework, no build step: the constraint (no Node) became a feature — the app is trivially hackable, loads instantly, and will still build in ten years.
- Data-driven lessons: adding a unit is pure data entry; six lessons and all exercises fall out of one declarative object.
- WebView over TWA/PWA: a real sideloadable APK with no hosting requirement, plus native speech recognition — the one thing a plain WebView cannot do via web APIs — solved with a 150-line bridge.
- Committed keystore: heresy for a store app, correct for a personal sideload — update continuity beats secrecy of a key that guards nothing.
- Honest metrics: 'words I know' counts only spaced-recall survivors, and reviews count toward the daily goal, so the streak rewards real practice.

Roadmap

- Phase 4 (in progress): tune lesson length, difficulty and review pacing based on daily use.
- DeepSeek integration (reserved slot): 'explain my mistake', fresh practice-sentence generation from known words, and a tutor chat — all optional extras layered on the offline core, with the API key stored locally.